Raft and KV-Raft: Consensus and State Machine Replication

Everton Romanzini Colombo
e257234@dac.unicamp.br

Student 2
student2@unicamp.br

***Abstract*—[TODO: 150–200 words: problem, algorithms (leader election and consensus/replication), containerized environment, metrics, and the main result (linearizable under faults; latency cost under loss/delay).]**

I. Introduction

[TODO: Motivation: why consensus matters; why Raft (understandable) and replicated state machines.] **[TODO:** Objectives: implement two complementary algorithms from the assignment (leader election [election theme] and consensus/replication [consensus theme]) and evaluate them experimentally.] **[TODO:** Contributions: (i) full Raft (2A–2C) with persistence and snapshots; (ii) linearizable KV with per-client deduplication; (iii) container packaging with TCP RPC and an HTTP API; (iv) test suite and reproducible experiments with linearizability checked by Porcupine.] **[TODO:** Paper organization (one paragraph).]

II. System Model

[TODO: State and justify each item below (explicit requirement in the assignment). Do not leave it as a bare list: justify the choices.]

A. Timing model

[TODO: Partially synchronous: no known bound on delay in steady state, but timeouts (500–700 ms election, 150 ms heartbeat) eventually hold, which is what provides liveness. Justify why not purely asynchronous.]

B. Channels

[TODO: Point-to-point channels over TCP (`net/rpc`), no ordering between concurrent requests, with loss/delay injected by `tc netem`; duplication and reordering are handled by idempotence (per-client `ClientId`, `Seq` dedup) and idempotent RPCs. In the test environment, `labrpc` simulates loss, delay and partitions.]

C. Failures

[TODO: Failure model: crash-stop and crash-recovery (state persisted on disk through `Persister`); network partitions; no Byzantine faults. Justify why persisted log plus snapshots suffice for recovery.]

D. Clocks, persistence and restart

[TODO: No synchronized clocks; timeouts use the local clock only as a heuristic. Persistence: `currentTerm`, `votedFor`, log and `lastIncludedIndex` in `raft.state`; snapshot in `kv.snap` (atomic write via temporary file + rename).]

E. Correctness properties

[TODO: Safety: at most one leader per term; Log Matching; if an entry is applied on one node it is applied at the same index on every node; KV linearizability. Liveness: after the network stabilizes and with a live majority, a leader is eventually elected and submitted commands are eventually committed.]

III. Architecture

[TODO: Figure/diagram: HTTP client → nodes (API :9000) and Raft mesh (:8000) between containers; persistent volumes.] **[TODO:** Describe `compose.yaml`: 3+ nodes, bridge network, ports 9000–9002 published, 8000 internal, one volume per node.] **[TODO:** Communication: binary RPC (`net/rpc`/gob) between peers; HTTP/JSON API for clients; `Clerk` on the client with retry and leader switching. Explain why the client (not the server) finds the leader.] **[TODO:** Observability: `/status` endpoint (term, leader, indexes, state and snapshot sizes) used by the experiments.] **[TODO:** Versions: Go, Docker/Compose, base image — fill in Table I.]

IV. Implemented Algorithms

[TODO: Make explicit that these are the two complementary algorithms: leader election (theme 1) and consensus/replication (theme 2).]

A. Leader election and terms

[TODO: Randomized timeouts, voting (log up-to-date rule, one vote per term), conversion to follower on a higher term; role of the observed `leaderId`.]

B. Log replication and consensus

[TODO: `AppendEntries` with `PrevLogIndex/Term`, retry with `XTerm/XIndex/XLen`, `commitIndex` advanced only for entries from the current term (Figure 8 of the paper), snapshots and `InstallSnapshot`.]

Table I
EXECUTION ENVIRONMENT

Item	Value
Hardware	**[TODO:** CPU, cores, RAM]
Operating system	**[TODO:** distribution, kernel]
Docker / Compose	**[TODO:** versions]
Go	**[TODO:** version]
Nodes	3, 5, 7 containers (same host)
Load	8 clients, 50 ops/client
Seeds	5–10 per configuration
Warm-up	1 discarded run (sweep)

C. Linearizable KV state machine

[TODO: Every operation (reads included) goes through the log; response cache keyed by `(ClientId, Seq)` for exactly-once application; replies matched by log index; snapshot of the map plus the cache.]

D. Reused code and modifications

[TODO: Mandatory citation: MIT 6.824 labs 2A–3B skeleton (`labrpc`, `labgob`, `porcupine`, tests) [1]; describe what was changed: the Raft/KV implementation, the `RPCClient` interface for real TCP, the on-disk `Persister`, the `/status` endpoint, the benchmark client and the experiment scripts.]

V. CORRECTNESS PROPERTIES AND TESTS

A. Safety arguments

[TODO: Sketch the proof: leader uniqueness per term; majority intersection; the election restriction guarantees the elected leader holds every committed entry; in-order application by index.]

B. Automated tests

[TODO: Describe the inherited suite (2A/2B/2C, 3A/3B) and our own tests: real TCP cluster, follower rejecting writes (HTTP 421), `/status` consistency. Report that everything passes (total time).] **[TODO:** Linearizability: Porcupine reconstructs a valid order for the history; 46/46 runs `linearizable=ok` in the experimental scenarios.]

VI. EXPERIMENTAL METHODOLOGY

[TODO: Describe the workload (50/50 read/write mix, 5 keys, `kvbench`), the per-run key namespace, and the fact that throughput is measured before the linearizability check.] **[TODO:** Metrics: throughput (ops/s), latency p50/p95/p99, time-to-elect, service recovery time (TTR-service), replica recovery time (TTR-replica), attempts per operation and `ErrWrongLeader` rate. Justify each one.] **[TODO:** Parameters varied at ≥ 3 levels: $N \in \{3, 5, 7\}$; delay/loss via `tc netem` (0, 50 ms, 100 ms; 0%, 2%, 5%); number of clients.] **[TODO:** Fault scenarios: leader crash, leader partition, full cluster restart.] **[TODO:** Reproducibility: `scripts/experiment.py` with recorded seeds, raw CSVs in `scripts/experiments/`, analysis in `scripts/analysis.ipynb`. Exact commands.]

Table II
THROUGHPUT AND LATENCY (MEDIAN OVER SEEDS)

Scenario	ops/s	p50	p95	p99
Normal	1633	4.5	7.4	8.8
`netem` 50 ms/2%	41	155	307	418
Leader crash (2 nodes)	2038	2.7	8.0	21.9
Partition (after heal)	1227	2.6	6.4	184

Table III
RECOVERY TIMES (MEAN $\pm$ STD)

Scenario	election	TTR-service	TTR-replica
Leader crash	507 ± 114 ms	1868 ± 32 ms	468 ± 124 ms
Partition	349 ± 185 ms	1632 ± 404 ms	not applicable

VII. RESULTS

[TODO: Replace the values below with the final version and generate the figures with `scripts/analysis.ipynb`.]

A. Normal operation

[TODO: Describe throughput and percentiles; explain that the 2% retry in the baseline is one extra attempt per client to discover the leader (8/400), not steady-state retrying.]

B. Delay and loss

[TODO: Discussion: throughput drop and p99 increase; why the ceiling is dominated by the election timeout and retransmissions; linearizability is preserved.]

C. Leader crash

[TODO: Time-to-elect $\approx$ election timeout; TTR-service dominated by the client HTTP timeout ($\sim$2 s); TTR-replica measures recovery via log/snapshot.]

D. Partition and the disruptive-server problem

[TODO: Inflated terms, churn, attempts per operation; explain that without PreVote the isolated node exiles itself; treat it as a limitation and future work.]

E. Scalability

[TODO: Results with $N = 3, 5, 7$: throughput and latency; discuss why the leader serializes and where replication cost would show up (message counts).]

VIII. LIMITATIONS AND THREATS TO VALIDITY

[TODO: Containers on one host are not independent failure domains; variability from short runs; no batching; no PreVote; no message/byte counters; Docker Desktop environment.]

IX. CONCLUSION AND FUTURE WORK

[TODO: Recap: two algorithms, correctness under faults, performance cost. Future work: PreVote/check-quorum, batching, lease-based reads, incremental snapshots, message accounting, execution across multiple machines.]

REFERENCES

[1] R. Morris, F. Kaashoek *et al.*, "6.824 distributed systems: Labs 2 and 3," MIT Computer Science and Artificial Intelligence Laboratory, 2020, skeleton for `labrpc`, `labgob`, `porcupine` and the test suite reused in this work. [Online]. Available: https://pdos.csail.mit.edu/6.824/2020